

LAPORAN PRAKTIKUM STRUKTUR DATA
PEKAN 9 SEARCHING (BFS DAN DFS)



Disusun Oleh :

DIKA GIOWANDA

NIM 2311533025

Dosen Pengampu :

DR. WAHYUDI, S.T, M.T

Asisten Praktikum :

SHERLY SUKMADIRA PUTRI

FAKULTAS TEKNOLOGI INFORMASI

DEPARTEMEN INFORMATIKA

UNIVERSITAS ANDALAS

PADANG, JUNI 2026

KATA PENGANTAR

Puji syukur penulis panjatkan ke hadirat Tuhan Yang Maha Esa atas limpahan rahmat, hidayah, dan karunia-Nya sehingga laporan praktikum Struktur Data tentang implementasi algoritma *Searching (Breadth-First Search dan Depth-First Search)* menggunakan bahasa pemrograman Java ini dapat diselesaikan dengan baik dan tepat waktu. Sholawat serta salam senantiasa tercurah kepada Nabi Muhammad SAW, keluarga, sahabat, dan seluruh pengikutnya hingga akhir zaman.

Laporan ini disusun sebagai salah satu bentuk pertanggungjawaban, refleksi, serta dokumentasi atas kegiatan praktikum yang telah dilaksanakan, dengan fokus pada penerapan konsep algoritma pencarian atau searching dalam struktur data. *Breadth-First Search (BFS)* dan *Depth-First Search (DFS)* merupakan dua algoritma traversal yang fundamental dalam menjelajahi struktur data *graf* dan *pohon*. Dalam praktikum ini, penulis mempelajari implementasi kedua algoritma tersebut menggunakan bahasa pemrograman Java, memahami perbedaan cara kerja *BFS* yang menggunakan pendekatan melebar dengan bantuan *queue* dan *DFS* yang menggunakan pendekatan mendalam dengan bantuan *stack*, serta menganalisis kompleksitas waktu dan ruang dari masing-masing algoritma dalam menyelesaikan permasalahan pencarian.

Penyusunan laporan ini tidak mungkin terwujud tanpa bantuan dan dukungan dari berbagai pihak. Oleh karena itu, penulis mengucapkan terima kasih yang tulus kepada dosen pembimbing yang telah memberikan arahan, bimbingan, dan masukan yang berharga selama proses praktikum, serta kepada rekan-rekan mahasiswa yang turut berkontribusi dalam diskusi dan kolaborasi.

Penulis menyadari bahwa laporan ini masih jauh dari sempurna. Oleh karena itu, kritik dan saran yang membangun sangat diharapkan demi perbaikan di masa mendatang. Semoga laporan ini dapat memberikan manfaat bagi pembaca, khususnya dalam memahami konsep dan implementasi algoritma *Breadth-First Search* dan *Depth-First Search* dalam bahasa pemrograman Java.

Akhir kata, penulis berharap semoga ilmu yang diperoleh melalui praktikum ini dapat bermanfaat dan diterapkan dengan baik.

Padang, 2026

Penyusun

DAFTAR ISI

KATA PENGANTAR.....	ii
DAFTAR ISI.....	iii
BAB 1 PENDAHULUAN.....	1
1.1 Pengertian Praktikum.....	1
1.2 Tujuan Praktikum.....	1
1.3 Persyaratan Praktikum.....	1
1.4 Tempat dan Waktu Praktikum.....	2
1.5 Manfaat Praktikum.....	2
BAB 2 PEMBAHASAN PRAKTIKUM.....	3
2.1 Pengertian Pemrograman Searching (BFS dan DFS).....	3
2.2 Langkah-Langkah Pengerjaan Praktikum Pekan 9.....	5
BAB 3 PENUTUP.....	20
3.1 Kesimpulan.....	20
3.2 Saran.....	20
DAFTAR PUSTAKA.....	21
LAPORAN TUGAS.....	22

BAB 1

PENDAHULUAN

1.1 Pengertian Praktikum

Praktikum Struktur Data adalah kegiatan pembelajaran yang dilakukan di laboratorium komputer untuk mengasah keterampilan mahasiswa dalam memahami serta menerapkan konsep pemrograman Java. Kegiatan ini tidak hanya menekankan pada penguasaan teori, tetapi juga pada latihan penyusunan kode program, pengujian, hingga analisis hasil eksekusi. Praktikum dipandang sebagai wahana latihan yang menjembatani pemahaman konseptual dengan kemampuan teknis pemrograman.

1.2 Tujuan Praktikum

Tujuan dari pelaksanaan praktikum ini antara lain sebagai berikut :

1. Membantu mahasiswa memahami konsep dasar pemrograman Struktur Data melalui penerapan langsung.
2. Melatih kemampuan menulis, mengompilasi, dan mengeksekusi program dengan mengikuti aturan sintaksis Java.
3. Memahami tipe data dasar hingga kompleks, termasuk konsep penyimpanan data.
4. Mampu menerapkan struktur data linear (*Linked List, Stack, Queue*) dan dalam bahasa pemrograman
5. Membiasakan mahasiswa bekerja sistematis dalam menyusun laporan yang memuat analisis hasil praktikum.
6. Menanamkan sikap teliti, disiplin, serta tanggung jawab dalam melaksanakan kegiatan laboratorium.
7. Mengetahui dan mengaplikasikan *ADT, ArrayList, Stack, Queue, Single Linked List, Double Linked List, Sorting 1 (Insertion, selection, bubble), Sorting 2 (shell, quick, merge), dan Searching (BFS dan DFS)*.

1.3 Persyaratan Praktikum

Agar praktikum berjalan lancar, mahasiswa perlu memenuhi beberapa persyaratan berikut:

1. Telah mengikuti perkuliahan teori Struktur Data sebagai dasar pemahaman.
2. Membawa perlengkapan yang diperlukan, antara lain laptop atau komputer yang sudah terpasang *Java Development Kit (JDK)* dan *Integrated Development Environment (IDE)* atau *Eclips* yang direkomendasikan.
3. Mengikuti setiap sesi praktikum sesuai jadwal yang ditetapkan dan hadir minimal sesuai ketentuan program studi.
4. Mematuhi tata tertib laboratorium, termasuk menjaga keamanan data, perangkat, serta lingkungan kerja.
5. Menyusun laporan praktikum dengan format dan aturan yang telah ditetapkan dalam pedoman ini.

1.4 Waktu dan Tempat Praktikum

Pelaksanaan praktikum Java mengikuti kalender akademik yang berlaku pada program studi. Setiap sesi praktikum dilaksanakan sesuai jadwal yang ditentukan oleh dosen pengampu. Tempat kegiatan umumnya berlangsung di laboratorium komputer, namun pada kondisi tertentu dapat dilaksanakan secara mandiri dengan perangkat masing-masing, selama memenuhi syarat teknis yang ditetapkan.

1.5 Manfaat Praktikum

Manfaat praktikum Struktur Data meliputi kemampuan memahami cara menyimpan data secara terstruktur, rapi, dan efisien, sehingga mengurangi penggunaan memori dan mempercepat akses data. Meningkatkan efisiensi penyimpanan dan manipulasi data, mengoptimalkan kecepatan algoritma, serta mempermudah penyelesaian masalah pemrograman yang kompleks. Selain itu, praktikum ini juga membantu membangun fondasi untuk mengembangkan aplikasi lintas platform yang lebih kompleks di berbagai bidang teknologi.

BAB 2

PEMBAHASAN PRAKTIKUM

2.1 Pengertian Pemrograman Searching (BFS dan DFS)

Breadth-First Search (BFS) dan *Depth-First Search (DFS)* adalah algoritma penelusuran (*traversal*) yang digunakan untuk mengunjungi, mencari, atau menjelajahi semua simpul (*node*) dalam struktur data pohon (*tree*) atau *graf* (*graph*). Keduanya membedakan diri dari cara arah dan prioritas kunjungannya.

Berikut adalah penjelasan dan perbandingan rinci untuk kedua algoritma tersebut pada praktikum struktur data:

1. Breadth-First Search (BFS)

BFS melakukan pencarian secara melebar atau tingkat per tingkat (*level-by-level*). Algoritma ini akan mengunjungi satu simpul, lalu mengunjungi semua simpul tetangganya secara horizontal terlebih dahulu sebelum turun ke tingkat (*level*) selanjutnya.

Struktur Data: Menggunakan struktur data antrian (*queue*) dengan prinsip *First In, First Out (FIFO)* untuk menyimpan simpul yang akan dikunjungi.

Kelebihan: Menjamin ditemukannya jalur terpendek (*shortest path*) pada *graph* yang tidak berbobot.

Kekurangan: Membutuhkan memori yang lebih besar karena harus menyimpan referensi semua simpul dalam satu tingkat sebelum pindah ke tingkat berikutnya.

2. Depth-First Search (DFS)

DFS melakukan pencarian secara mendalam. Algoritma ini menelusuri satu cabang sebuah *graf* atau pohon sampai selesai hingga menemukan solusi atau jalan buntu, kemudian melakukan *backtracking* (merunut-balik) untuk mencoba cabang lainnya.

Struktur Data: Menggunakan struktur data tumpukan (*stack*) dengan prinsip *Last In, First Out (LIFO)*.

Kelebihan: Sangat hemat memori karena hanya perlu mengingat jalur spesifik dari akar hingga ke simpul yang sedang ditelusuri.

Kekurangan: Tidak selalu menghasilkan jalur terpendek dan bisa terjebak pada jalur yang sangat dalam jika tidak ada batas kedalaman.

Komponen utama algoritma *Breadth-First Search (BFS)* dan *Depth-First Search (DFS)* dalam praktikum struktur data mencakup representasi *graf/tree*, memori penyimpanan (*Queue* atau *Stack*), serta pelacak status *node*. Keduanya digunakan untuk menelusuri data secara terstruktur.

Berikut adalah rincian komponen utamanya:

1. Representasi Graf atau Tree

Setiap pencarian membutuhkan struktur data dasar untuk memetakan hubungan antar data.

Vertex (Simpul): Merepresentasikan data atau titik lokasi.

Edge (Sisi): Merepresentasikan jalur atau relasi antar simpul. Sering direpresentasikan menggunakan *Adjacency List* (daftar ketetanggaan) atau *Adjacency Matrix* (matriks ketetanggaan).

2. Memori Penyimpanan (Fringe/Border)

Komponen ini bertugas menampung simpul-simpul yang akan dikunjungi selanjutnya.

Queue (Antrean): Digunakan oleh *BFS* dengan prinsip *First-In First-Out (FIFO)*. Node yang pertama kali dimasukkan akan diproses lebih dulu, memastikan pencarian melebar ke setiap level secara berurutan.

Stack (Tumpukan): Digunakan oleh *DFS* dengan prinsip *Last-In First-Out (LIFO)* atau melalui fungsi rekursi. Algoritma ini menelusuri jalur hingga ke titik terdalam sebelum melakukan *backtracking* (mundur).

3. Pelacak Status Kunjungan (*Visited Array*)

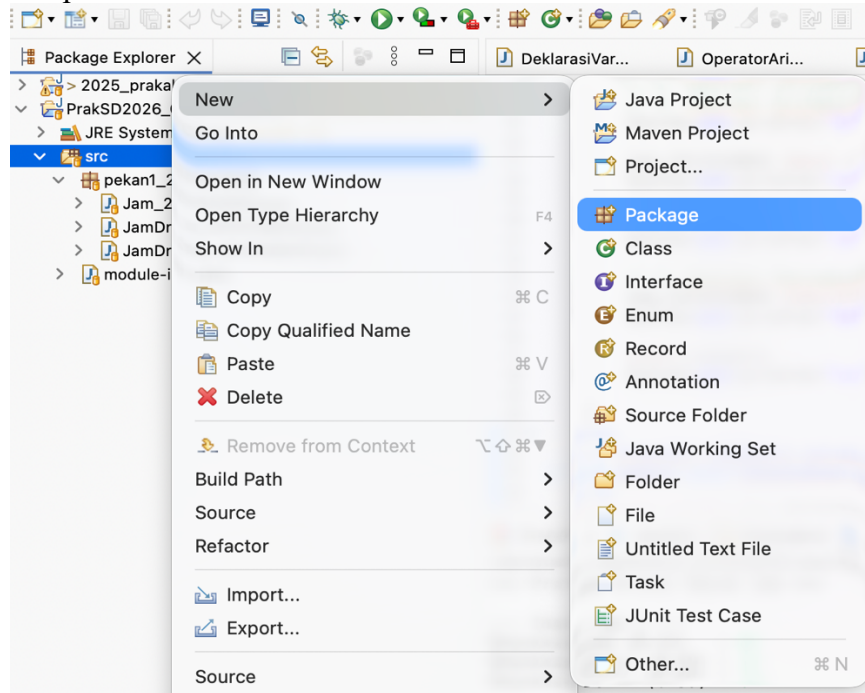
Array Boolean: Digunakan untuk menandai apakah suatu simpul sudah pernah dikunjungi (*true*) atau belum (*false*).

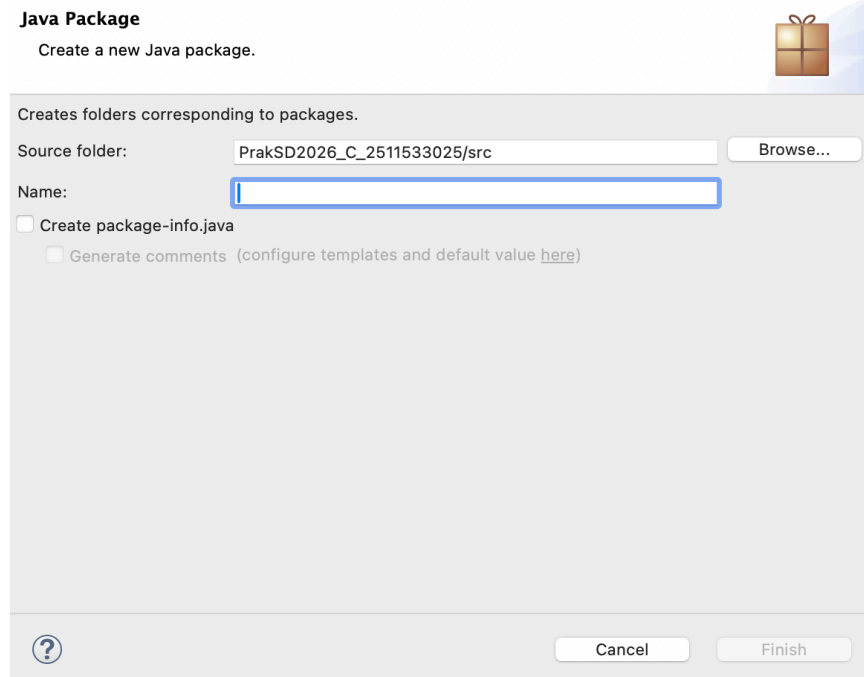
Fungsi: Komponen ini sangat krusial, terutama pada *graf* yang memiliki siklus (*cycle*), untuk mencegah program mengalami *infinite loop* atau memproses node yang sama berulang kali.

2.2 Langkah-Langkah Pengerjaan Praktikum Pekan 9

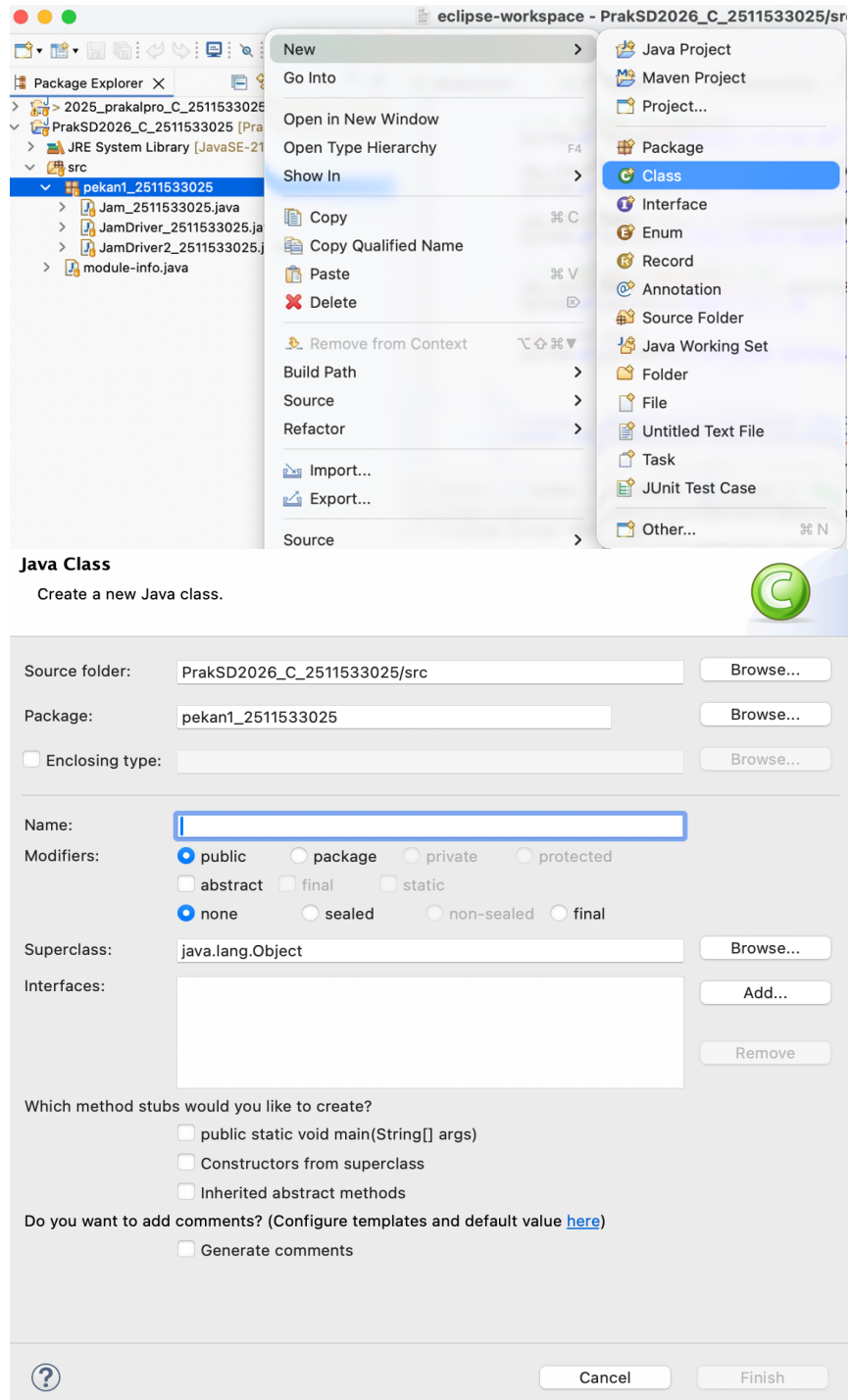
Sebelum ke langkah-langkah pengerjaan contoh proyek, saya akan menjelaskan terlebih dahulu gimana caranya menambahkan proyek nya terlebih dahulu :

1. Klik kanan pada folder src, pilih new, pilih package. Nanti akan muncul halaman Java Package, cukup masukan nama, nama wajib no spasi. Klik finish.





2. Klik kanan pada folder package, pilih new, pilih class. Nanti akan muncul halaman Java Class, cukup masukan nama class sesuai projek yang akan dibuat, nama wajib no spasi. Klik finish.



Catatan :

Jika kita ingin membuat proyek baru lagi, maka cukup buat folder class yang baru.

- A. Baiklah karena proses menambahkan proyek sudah dijelaskan, selanjutnya langkah-langkah pengerjaan proyek pekan 9:

a. Class Node_2511533025

1. Kode mendefinisikan sebuah kelas bernama Node_2511533025 yang berfungsi sebagai blok penyusun dasar untuk struktur data Binary Tree (pohon biner) dalam bahasa pemrograman Java. Secara sederhana, kelas ini menyimpan sebuah data berupa angka (int) dan memiliki dua "penunjuk" atau referensi ke cabang sebelah kiri (left) dan kanan (right) untuk menghubungkan satu node dengan node lainnya. Di dalamnya juga terdapat metode constructor untuk membuat objek baru, serta metode getter dan setter yang memungkinkan pengguna untuk mengambil atau mengubah nilai data serta mengatur hubungan antar node dalam struktur pohon tersebut.

```
1 package Pekan9_2511533025;
2 public class Node_2511533025 {
3     int data_3025;
4     Node_2511533025 left_3025;
5     Node_2511533025 right_3025;
6
7     public Node_2511533025(int data_3025) {
8         this.data_3025 = data_3025;
9         left_3025 = null;
10        right_3025 = null;
11    }
12
13    public void setLeft(Node_2511533025 node_3025) {
14        if (left_3025 == null)
15            left_3025 = node_3025;
16    }
17
18    public void setRight(Node_2511533025 node_3025) {
19        if (right_3025 == null)
20            right_3025 = node_3025;
21    }
22
23    public Node_2511533025 getLeft() {
24        return left_3025;
25    }
26
27    public Node_2511533025 getRight() {
28        return right_3025;
29    }
30
31    public int getData() {
32        return data_3025;
33    }
34
35    public void setData(int data) {
36        this.data_3025 = data;
37    }
38}
```

2. Kode ini berisi tiga metode rekursif untuk melakukan penelusuran (traversal) pada struktur data pohon biner, yaitu Preorder, Postorder, dan Inorder. Fungsi-fungsi ini digunakan untuk mengunjungi dan menampilkan data dari setiap titik (node) dalam urutan tertentu: Preorder mencetak data induk sebelum cabangnya, Postorder mencetak data induk setelah semua cabangnya selesai diproses, dan Inorder mencetak data secara berurutan dari cabang kiri, ke induk, lalu ke cabang kanan. Secara sederhana, rangkaian kode ini berfungsi untuk

membaca dan menampilkan seluruh isi pohon biner secara sistematis sesuai pola urutan yang diinginkan.

```

39 void printPreorder(Node_2511533025 node_3025) {
40     if (node_3025 == null)
41         return;
42     System.out.print(node_3025.data_3025 + " ");
43     printPreorder(node_3025.left_3025);
44     printPreorder(node_3025.right_3025);
45 }
46
47 void printPostorder(Node_2511533025 node_3025) {
48     if (node_3025 == null)
49         return;
50     printPostorder(node_3025.left_3025);
51     printPostorder(node_3025.right_3025);
52     System.out.print(node_3025.data_3025 + " ");
53 }
54
55 void printInorder(Node_2511533025 node_3025) {
56     if (node_3025 == null)
57         return;
58     printInorder(node_3025.left_3025);
59     System.out.print(node_3025.data_3025 + " ");
60     printInorder(node_3025.right_3025);
61 }

```

3. Kode ini berfungsi untuk menampilkan struktur Binary Tree secara visual ke konsol dengan format yang rapi dan mudah dibaca. Fungsi ini bekerja secara rekursif, di mana ia menelusuri setiap cabang pohon (kanan dan kiri) sambil menyusun simbol karakter seperti garis lurus, sudut, dan spasi sebagai "prefix" untuk menunjukkan hierarki dan hubungan antar node. Hasil akhirnya adalah representasi grafis dari data di dalam pohon yang memperlihatkan posisi setiap elemen terhadap elemen lainnya.

```

63 public String print() {
64     return this.print("", true, "");
65 }
66
67 public String print(String prefix_3025, boolean isTail_3025, String sb_3025) {
68     if (right_3025 != null) {
69         right_3025.print(prefix_3025 + (isTail_3025 ? "| " : " "), false, sb_3025);
70     }
71     System.out.println(prefix_3025 + (isTail_3025 ? "\\-- " : "/-- ") + data_3025);
72     if (left_3025 != null) {
73         left_3025.print(prefix_3025 + (isTail_3025 ? " " : "| "), true, sb_3025);
74     }
75     return sb_3025;
76 }
77 }

```

b. Class BTree_2511533025

1. Kode ini mendefinisikan sebuah kelas Java bernama BTree_2511533025 yang merepresentasikan struktur data pohon biner (Binary Tree). Di dalamnya, terdapat variabel root_3025 sebagai titik awal pohon, sebuah konstruktor untuk membuat pohon baru yang awalnya kosong (null), serta sebuah metode search yang berfungsi untuk mencari data angka tertentu di dalam pohon tersebut dengan memanggil fungsi pencarian lain yang bekerja secara rekursif. Secara sederhana, kode ini

adalah kerangka dasar untuk membangun dan mencari data dalam struktur pohon digital.

```
1 package Pekan9_2511533025;
2 public class BTree_2511533025 {
3     private Node_2511533025 root_3025;
4     private Node_2511533025 currentNode;
5
6     public BTree_2511533025() {
7         root_3025 = null;
8     }
9
10    public boolean search(int data_3025) {
11        return search(root_3025, data_3025);
12    }
13 }
```

2. Kode tersebut merupakan fungsi pencarian rekursif yang digunakan untuk menemukan nilai tertentu (data_3025) di dalam struktur data pohon (seperti Binary Tree). Fungsi ini bekerja dengan cara memeriksa apakah data pada simpul (node) saat ini sesuai dengan nilai yang dicari; jika cocok, fungsi langsung mengembalikan nilai true. Jika tidak cocok, fungsi akan menelusuri cabang kiri secara rekursif, diikuti oleh cabang kanan jika nilai belum ditemukan di sisi kiri. Apabila seluruh simpul di bawah struktur tersebut telah diperiksa dan nilai target tetap tidak ditemukan, maka fungsi akan mengembalikan nilai false.

```
14 private boolean search(Node_2511533025 node_3025, int data_3025) {
15     if (node_3025.getData() == data_3025) {
16         return true;
17     }
18     if (node_3025.getLeft() != null) {
19         if (search(node_3025.getLeft(), data_3025)) {
20             return true;
21         }
22     }
23     if (node_3025.getRight() != null) {
24         if (search(node_3025.getRight(), data_3025)) {
25             return true;
26         }
27     }
28     return false;
29 }
```

3. Kode tersebut berisi tiga metode publik—printInorder(), printPreOrder(), dan printPostOrder()—yang berfungsi sebagai pintu masuk (wrapper) untuk mencetak seluruh isi struktur data pohon (tree). Setiap metode memanggil fungsi rekursif pada objek root_3205 untuk menelusuri dan menampilkan data berdasarkan urutan traversal tertentu, yaitu urutan tengah (Inorder), urutan awal (Preorder), atau urutan akhir (Postorder). Secara sederhana, kode ini memudahkan pengguna untuk memulai proses pencetakan pohon tanpa harus memasukkan parameter akar (root) secara manual.

```

26 public void printInorder() {
27     root_3025.printInorder(root_3025);
28 }
29
30 public void printPreOrder() {
31     root_3025.printPreorder(root_3025);
32 }
33
34 public void printPostOrder() {
35     root_3025.printPostorder(root_3025);
36 }

```

4. Kode ini berisi tiga metode Java untuk mengelola struktur data pohon (tree): `getRoot()` berfungsi untuk mengambil atau mengakses simpul akar (root), `isEmpty()` digunakan untuk memeriksa apakah pohon tersebut kosong dengan mengecek apakah akarnya bernilai null, dan `countNodes()` berfungsi untuk menghitung total jumlah simpul yang ada di dalam pohon dengan memanggil metode bantuan. Secara keseluruhan, potongan kode ini menyediakan fungsionalitas dasar untuk mengakses informasi mengenai status dan ukuran dari sebuah pohon.

```

38 public Node_2511533025 getRoot() {
39     return root_3025;
40 }
41 public boolean isEmpty() {
42     return root_3025 == null;
43 }
44 public int countNodes() {
45     return countNodes(root_3025);
46 }

```

5. Kode tersebut adalah fungsi rekursif bernama `countNodes` yang bertujuan untuk menghitung jumlah total node (titik) dalam sebuah struktur data pohon (tree). Cara kerjanya adalah dengan memeriksa apakah node saat ini ada atau kosong; jika kosong, ia mengembalikan nilai 0, namun jika ada, ia akan menghitung node tersebut sebagai 1 dan menjumlahkannya dengan hasil perhitungan dari cabang kiri dan cabang kanan secara berulang hingga seluruh bagian pohon terhitung. Hasil akhirnya adalah jumlah akumulasi dari seluruh node yang terhubung dalam pohon tersebut.

```

47 private int countNodes(Node_2511533025 node_3025) {
48     int count_3025 = 1;
49     if (node_3025 == null) {
50         return 0;
51     } else {
52         count_3025 += countNodes(node_3025.getLeft());
53         count_3025 += countNodes(node_3025.getRight());
54         return count_3025;
55     }
56 }

```

6. Kode ini berisi sekumpulan metode untuk mengelola struktur data berbasis node, seperti pohon (tree) atau graf. Metode print

berfungsi untuk menampilkan isi struktur data mulai dari akar (root), sementara metode lainnya merupakan getter dan setter yang digunakan untuk mengambil atau memperbarui posisi node saat ini (currentNode) serta menentukan node utama yang menjadi akar (root_3025). Secara keseluruhan, kode ini berperan dalam mengatur akses dan navigasi antar elemen di dalam struktur tersebut.

```

57 public void print() {
58     root_3025.print();
59 }
60 public Node_2511533025 getCurrent() {
61     return currentNode;
62 }
63 public void setCurrent(Node_2511533025 node_3025) {
64     this.currentNode = node_3025;
65 }
66 public void setRoot(Node_2511533025 root_3025) {
67     this.root_3025 = root_3025;
68 }
69 }

```

c. Class BtreeaDriver_2511533025

1. Kode ini merupakan program utama (driver) yang berfungsi untuk menginisialisasi dan memulai struktur data pohon biner (Binary Tree). Di dalam metode main, program membuat objek pohon baru, menampilkan jumlah simpul (node) awal ke layar menggunakan metode countNodes(), lalu membuat sebuah simpul baru dengan nilai "1" yang dipersiapkan sebagai akar (root) dari pohon tersebut.

```

1 package Pekan9_2511533025;
2
3 public class BtreeDriver_2511533025 {
4
5     public static void main(String[] args) {
6         //Membuat Pohon
7         BTree_2511533025 tree_3025 = new BTree_2511533025();
8         System.out.print("Jumlah Simpul awal pohon: ");
9         System.out.println(tree_3025.countNodes());
10        //menambahkan simpul data 1
11        Node_2511533025 root_3025 = new Node_2511533025(1);

```

2. Kode ini untuk menginisialisasi struktur data pohon (tree) dalam bahasa pemrograman Java dengan menetapkan root_3025 sebagai akar dari objek tree_3025, kemudian mencetak jumlah simpul awal yang ada. Selanjutnya, kode melakukan instansiasi delapan objek simpul baru (node2 hingga node9) menggunakan kelas Node_2511533025 dengan memberikan nilai numerik yang berurutan sebagai argumen konstruktornya. Secara keseluruhan, bagian kode ini merupakan tahap persiapan untuk membangun struktur hierarki pohon yang lebih kompleks dengan menciptakan node-node yang nantinya akan saling dihubungkan.

```

12 //meniadikan simpul 1 sebagai root
13 tree_3025.setRoot(root_3025);
14 System.out.println("Jumlah simpul jika hanya ada root");
15 System.out.println(tree_3025.countNodes());
16 Node_2511533025 node2_3025 = new Node_2511533025(2);
17 Node_2511533025 node3_3025 = new Node_2511533025(3);
18 Node_2511533025 node4_3025 = new Node_2511533025(4);
19 Node_2511533025 node5_3025 = new Node_2511533025(5);
20 Node_2511533025 node6_3025 = new Node_2511533025(6);
21 Node_2511533025 node7_3025 = new Node_2511533025(7);
22 Node_2511533025 node8_3025 = new Node_2511533025(8);
23 Node_2511533025 node9_3025 = new Node_2511533025(9);

```

3. Kode tersebut berfungsi untuk menyusun struktur hirarki sebuah pohon biner (binary tree) dengan cara menghubungkan berbagai objek simpul (node) satu sama lain. Setiap baris instruksi menentukan hubungan antara "induk" dan "anak", di mana metode setLeft digunakan untuk menempatkan simpul di sisi kiri dan setRight untuk sisi kanan. Secara keseluruhan, rangkaian kode ini membangun sebuah pohon yang berakar pada root_3025 dan memiliki beberapa cabang di bawahnya hingga mencapai level tertentu.

```

24 root_3025.setLeft(node2_3025);
25 node2_3025.setLeft(node4_3025);
26 node2_3025.setRight(node5_3025);
27 node4_3025.setRight(node8_3025);
28 root_3025.setRight(node3_3025);
29 node3_3025.setLeft(node6_3025);
30 node3_3025.setRight(node7_3025);
31 node6_3025.setLeft(node9_3025);

```

4. Kode ini untuk mengelola dan menampilkan informasi dari struktur data pohon (tree) bernama tree_3025. Program ini dimulai dengan menetapkan posisi aktif ke akar (root) pohon dan mencetak datanya, menghitung jumlah total simpul yang ada, serta melakukan penelusuran pohon menggunakan metode In-order, Pre-order, dan Post-order. Terakhir, kode tersebut memanggil fungsi untuk menampilkan representasi visual dari struktur pohon secara keseluruhan ke layar.

```

32 //Set root
33 tree_3025.setCurrent(tree_3025.getRoot());
34 System.out.println("menampilkan simpul terakhir: ");
35 System.out.println(tree_3025.getCurrent().getData());
36 System.out.println("Jumlah simpul; setelah simpul 7 ditambahkan");
37 System.out.println(tree_3025.countNodes());
38 System.out.println("InOrder: ");
39 tree_3025.printInorder();
40 System.out.println("\nPreorder: ");
41 tree_3025.printPreOrder();
42 System.out.println("\nPostorder : ");
43 tree_3025.printPostOrder();
44 System.out.println("\nMenampilkan simpul dalam bentuk pohon");
45 tree_3025.print();
46
47 }
48
49 }

```

d. Class GraphTraversal_2511533025

1. Kode ini untuk mengelola struktur data graf tak berarah, di mana fungsi utamanya adalah untuk membangun hubungan antar titik (node). Di dalamnya terdapat metode `addEdge` yang secara otomatis mendaftarkan dua buah titik ke dalam peta graf (menggunakan `HashMap`) jika belum ada, lalu menghubungkan keduanya dengan cara menambahkan nama masing-masing titik ke dalam daftar tetangga satu sama lain. Karena ini adalah graf tak berarah, koneksi dibuat dua arah agar setiap titik dapat mengetahui siapa saja tetangganya secara timbal balik.

```
1 package Pekan9_2511533025;
2 import java.util.*;
3
4 public class GraphTraversal_2511533025 {
5     private Map<String, List<String>> graph_3025 = new HashMap<>();
6
7     // Menambahkan edge (graf tak berarah)
8     public void addEdge(String node1_3025, String node2_3025) {
9         graph_3025.putIfAbsent(node1_3025, new ArrayList<>());
10        graph_3025.putIfAbsent(node2_3025, new ArrayList<>());
11        graph_3025.get(node1_3025).add(node2_3025);
12        graph_3025.get(node2_3025).add(node1_3025);
13    }
14 }
```

2. Kode ini berfungsi untuk menampilkan representasi graf dalam bentuk Adjacency List ke layar. Melalui metode `printGraph()`, program melakukan iterasi pada setiap simpul (node) yang tersimpan dalam Map `graph_3025`, lalu mencetak nama simpul tersebut diikuti dengan daftar tetangga (neighbors) yang terhubung dengannya. Setiap tetangga dipisahkan oleh tanda koma menggunakan fungsi `String.join`, sehingga pengguna dapat dengan mudah melihat struktur hubungan antar titik dalam graf tersebut.

```
14
15 // Menampilkan graf awal
16 public void printGraph() {
17     System.out.println("Graf Awal (Adjacency List):");
18     for (String node_3025 : graph_3025.keySet()) {
19         System.out.print(node_3025 + " -> ");
20         List<String> neighbors_3025 = graph_3025.get(node_3025);
21         System.out.println(String.join(", ", neighbors_3025));
22     }
23     System.out.println();
24 }
25 }
```

3. Kode tersebut merupakan fungsi utama untuk memulai algoritma Depth First Search (DFS) secara rekursif pada sebuah graf. Fungsi ini menginisialisasi sebuah `HashSet` bernama `visited_3025` untuk mencatat simpul yang telah dikunjungi agar tidak terjadi pengulangan atau loop tak terbatas, lalu memanggil fungsi pembantu `dfsHelper` untuk melakukan penelusuran mendalam mulai dari simpul `start_3025`. Secara keseluruhan, metode ini berfungsi sebagai titik masuk (entry point) yang menyiapkan struktur data yang diperlukan sebelum proses pencarian rekursif dijalankan.

```

25
26 // DFS rekursif
27 public void dfs(String start_3025) {
28     Set<String> visited_3025 = new HashSet<>();
29     System.out.println("Penelusuran DFS:");
30     dfsHelper(start_3025, visited_3025);
31     System.out.println();
32 }
33

```

4. Kode ini merupakan implementasi metode pembantu (helper method) untuk algoritma Depth-First Search (DFS) pada sebuah graf menggunakan bahasa pemrograman Java. Fungsi dfsHelper ini bekerja secara rekursif untuk menjelajahi simpul-simpul graf; ia pertama-tama memeriksa apakah simpul saat ini (current_3025) sudah pernah dikunjungi menggunakan himpunan visited_3025 untuk mencegah pengulangan atau infinite loop. Jika belum, simpul tersebut ditandai sebagai sudah dikunjungi, dicetak ke layar, dan kemudian fungsi memanggil dirinya sendiri secara rekursif untuk setiap tetangga (neighbor_3025) dari simpul tersebut yang diambil dari daftar kedekatan (graph_3025).

```

34 private void dfsHelper(String current_3025, Set<String> visited_3025) {
35     if (visited_3025.contains(current_3025)) return;
36     visited_3025.add(current_3025);
37     System.out.print(current_3025 + " ");
38     for (String neighbor_3025 : graph_3025.getDefault(current_3025, new ArrayList<>())) {
39         dfsHelper(neighbor_3025, visited_3025);
40     }
41 }

```

5. Kode ini mengimplementasikan algoritma Breadth-First Search (BFS) secara iteratif dalam bahasa pemrograman Java untuk menelusuri struktur graf. Fungsi ini bekerja dengan menggunakan sebuah antrian (Queue) untuk mengelola simpul yang akan dikunjungi, dimulai dari simpul start_3025, serta set visited_3025 untuk melacak simpul-simpul yang sudah diproses agar tidak terjadi pengulangan atau iterasi tanpa akhir. Di dalam blok perulangan while, program mengambil simpul dari depan antrian, mencetaknya, lalu memeriksa semua tetangga yang terhubung melalui objek graph_3025; jika tetangga tersebut belum pernah dikunjungi, maka simpul tersebut akan dimasukkan ke dalam antrian dan ditandai sebagai sudah dikunjungi guna memastikan penelusuran dilakukan secara melebar lapis demi lapis dari titik awal.

```

42 // BFS iteratif
43 public void bfs(String start_3025) {
44     Set<String> visited_3025 = new HashSet<>();
45     Queue<String> queue_3025 = new LinkedList<>();
46     queue_3025.add(start_3025);
47     visited_3025.add(start_3025);
48     System.out.println("Penelusuran BFS:");
49     while (!queue_3025.isEmpty()) {
50         String current_3025 = queue_3025.poll();
51         System.out.print(current_3025 + " ");
52         for (String neighbor_3025 : graph_3025.getOrDefault(current_3025, new ArrayList<>())) {
53             if (!visited_3025.contains(neighbor_3025)) {
54                 queue_3025.add(neighbor_3025);
55                 visited_3025.add(neighbor_3025);
56             }
57         }
58     }
59     System.out.println();
60 }

```

6. Kode ini merupakan fungsi utama (main) dalam bahasa pemrograman Java yang bertujuan untuk mensimulasikan struktur data graf dan metode penelusurannya. Di dalamnya, sebuah objek graf dibuat, kemudian beberapa sisi (edge) ditambahkan untuk menghubungkan simpul A hingga E sesuai contoh skenario. Program tersebut kemudian mencetak struktur graf awal ke layar dan menjalankan dua algoritma penelusuran standar, yaitu Depth First Search (DFS) dan Breadth First Search (BFS), yang dimulai dari simpul "A" untuk menampilkan urutan kunjungan simpul-simpul dalam graf tersebut..

```

62 // Main
63 public static void main(String[] args) {
64     GraphTraversal_2511533025 graph_3025 = new GraphTraversal_2511533025();
65
66     // Contoh graf: A-B, A-C, B-D, B-E
67     graph_3025.addEdge("A", "B");
68     graph_3025.addEdge("A", "C");
69     graph_3025.addEdge("B", "D");
70     graph_3025.addEdge("B", "E");
71     // Cetak graf awal
72     System.out.println("Graf Awal adalah: ");
73     graph_3025.printGraph();
74     // Lakukan penelusuran
75     graph_3025.dfs("A");
76     graph_3025.bfs("A");
77 }
78
79 }

```

e. Hasil proyek pekan 9.

1. Output dari Class Node_2511533025, BTree_2511533025, dan BtreeDriver_2511533025.

```

Problems @ Javadoc Declaration Console X
<terminated> BtreeDriver_2511533025 [Java Application] /Appli
Jumlah Simpul awal pohon: 0
Jumlah simpul jika hanya ada root
1
menampilkan simpul terakhir:
1
Jumlah simpul; setelah simpul 7 ditambahkan
9
InOrder:
4 8 2 5 1 9 6 3 7
Preorder:
1 2 4 8 5 3 6 9 7
Postorder :
8 4 5 2 9 6 7 3 1
Dmenampilkan simpul dalam bentuk pohon
|           /-- 7
|         /-- 3
|         |   \-- 6
|         |   \-- 9
|       /-- 1
|       |   /-- 5
|       |   \-- 2
|       |   |   /-- 8
|       |   |   \-- 4

```

2. Output dari Class GraphTraversal_2511533025.

```

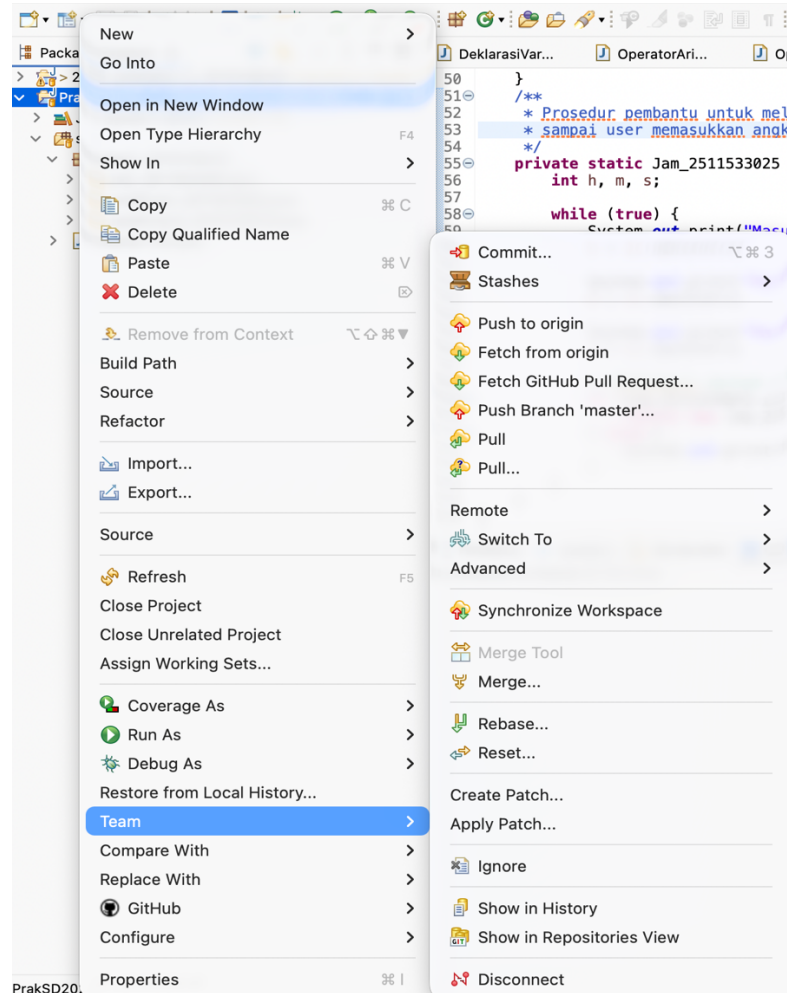
Problems @ Javadoc Declaration
<terminated> GraphTraversal_2511533025 [Ja
Garf Awal adalah:
Graf Awal (Adjacency List):
A -> B, C
B -> A, D, E
C -> A
D -> B
E -> B

Penelusuran DFS:
A B D E C
Penelusuran BFS:
A B C D E

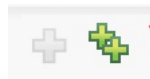
```

B. Langkah Penyimpanan

1. Sebelum kita masuk ke langkah, kita harus buat akun github dulu. Jika sudah buat akun github, baru bisa kita simpan di githubnya. Selanjutnya ikutin langkah ini. Tekan kanan mouse pada folder proyek, setelah itu pilih team, terus ke *commit*.



2. Lalu tekan tombol plus 2/double plus warna hijau jika ingin push semua kode, kalau hanya salah satu klik kode yang ingin di push, terus klik tanda plus single.

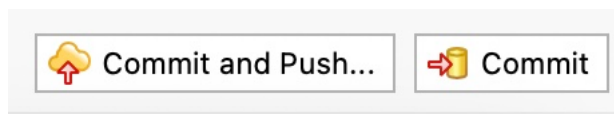


3. Lalu masukan pesan pada kolom *commit message*.

Commit Message

i Unborn branch: this commit will create the branch 'master'.

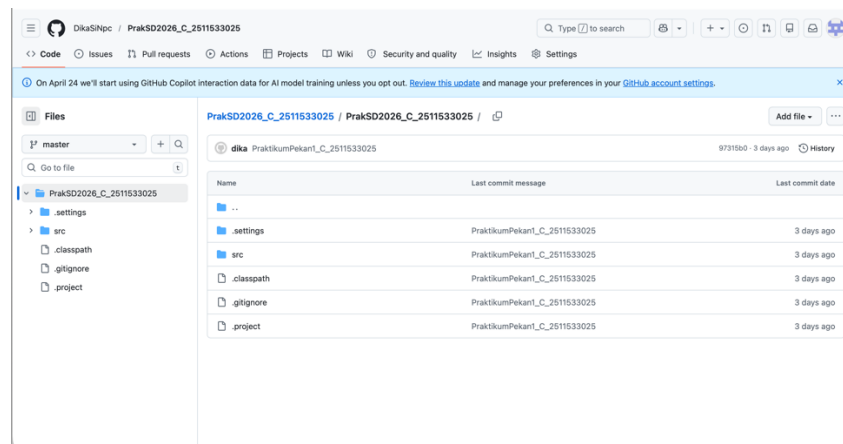
4. Setelah itu tekan *commit* and pus, lalu ikutin arahan untuk memasukkan nama dan pw akun git hub kalian, maka proyek kalian sudah tersimpan di akun github kalian.



5. Proyek yang sudah disimpan di akun proyek.



6. Gambar proyek apabila sudah masuk di akun github.



Sedikit tambahan jika ingin cek hasil program nya tekan tombol *run* warna hijau.



Catatan :

Jika kita ingin membuat proyek baru lagi, maka cukup buat folder class yang baru atau package yang baru.

BAB 3 PENUTUP

3.1 Kesimpulan

Berdasarkan hasil praktikum yang telah dilakukan, dapat disimpulkan bahwa *Breadth-First Search (BFS)* dan *Depth-First Search (DFS)* merupakan dua algoritma traversal fundamental yang digunakan untuk menjelajahi atau mencari elemen dalam struktur data *graf* dan pohon. *BFS* bekerja dengan pendekatan melebar (*level-by-level*) menggunakan bantuan struktur data *Queue*, sehingga sangat efektif untuk mencari jalur terpendek pada *graf* tanpa bobot, sedangkan *DFS* bekerja dengan pendekatan mendalam (*backtracking*) menggunakan bantuan *Stack*, yang lebih efisien untuk mengeksplorasi seluruh kemungkinan jalur atau mendeteksi siklus dalam *graf*. Penulis memahami bahwa pemilihan antara *BFS* dan *DFS* sangat bergantung pada karakteristik masalah dan struktur data yang dihadapi, di mana *BFS* memiliki kompleksitas ruang yang lebih besar karena menyimpan semua *node* pada level tertentu, sementara *DFS* cenderung lebih hemat memori namun berisiko mengalami *stack overflow* pada *graf* yang sangat dalam.

3.2 Saran

Untuk pengembangan selanjutnya, Disarankan agar praktikum tidak hanya memahami cara mengimplementasikan kode *BFS* dan *DFS*, tetapi juga memperdalam analisis terhadap kompleksitas waktu dan ruang dari kedua algoritma tersebut agar mampu memilih strategi pencarian yang paling optimal sesuai dengan kasus nyata. Praktikum selanjutnya dapat diperkaya dengan studi kasus yang lebih aplikatif, seperti implementasi *BFS* untuk pencarian rute terpendek pada sistem navigasi atau *DFS* untuk penyelesaian maze dan deteksi siklus, sehingga mahasiswa dapat melihat relevansi langsung materi ini dalam pengembangan perangkat lunak. Serta lebih memperhatikan penanganan *graf* berbobot, *graf* berarah, dan kondisi khusus.

DAFTAR PUSTAKA

- Sumber daring (website):

[1] R. Munir dan N. U. Maulidevi, "Breadth/Depth First Search (Bagian 1) (BFS/DFS)," Bahan Kuliah IF2211 Strategi Algoritmik, Program Studi Teknik Informatika, Sekolah Teknik Elektro dan Informatika ITB, 2021. [Online]. Tersedia: <https://informatika.stei.itb.ac.id/~rinaldi.munir/Stmik/2020-2021/BFS-DFS-2021-Bag1.pdf>. [Diakses: 13 Mei 2026].

[2] P. Vadapalli, "DFS in Data Structure: Depth-First Search Algorithm Explained," upGrad, 17 Jun. 2025. [Online]. Tersedia: <https://www.upgrad.com/blog/dfs-depth-first-traversal-in-data-structure/>. [Diakses: 13 Mei 2026].

[3] G. Yanuar, "Algoritma BFS dan DFS dalam Graph," Scribd, n.d. [Online]. Tersedia: <https://www.scribd.com/presentation/515295113/13-Struktur-Data-Algoritma-BFS-DFS>. [Diakses: 13 Mei 2026].

[4] M. A. Adiputra, "Penerapan Algoritma BFS dan DFS untuk Penjadwalan Rencana Studi," Makalah IF2211 Struktur Data, Sekolah Teknik Elektro dan Informatika, Institut Teknologi Bandung, 2018. [Online]. Tersedia: <https://informatika.stei.itb.ac.id/~rinaldi.munir/Stmik/2017-2018/Makalah/Makalah-IF2211-2018-040.pdf>. [Diakses: 13 Mei 2026].

LAPORAN TUGAS PRAKTIKUM PEKAN 9

1. Intruksi Tugas Praktikum Pekan 9

Deskripsi Tugas

Mahasiswa diminta untuk mengimplementasikan algoritma BFS (Breadth First Search) dan DFS (Depth First Search) menggunakan bahasa Java dengan antarmuka GUI. Studi kasus yang digunakan adalah pencarian jalur pada suatu peta lokasi yang direpresentasikan dalam bentuk Graph. Setiap mahasiswa wajib membuat peta lokasi yang berbeda dengan mahasiswa lain. Peta dapat berupa:

- Gedung Perkuliahan
- Fakultas
- Universitas
- Sekolah
- Rumah Sakit
- Mall
- Tempat Wisata
- Bandara
- Atau lokasi lainnya

Program harus mampu:

1. Menampilkan graph dalam bentuk visual
2. Menentukan titik awal (Start)
3. Menentukan titik tujuan (Goal)
4. Melakukan pencarian menggunakan BFS
5. Melakukan pencarian menggunakan DFS
6. Menampilkan jalur yang ditemukan
7. Menampilkan urutan simpul yang dikunjungi
8. Urutan node yang dikunjungi
9. Jumlah node yang dieksplorasi

Aturan Penamaan

1. Penamaan class dan file

Gunakan akhiran NIM lengkap pada nama class dan nama file.

Contoh:

ex : PetaKampus_2311532008

2. Penamaan Variabel

Gunakan 4 digit terakhir pada setiap nama variabel

ex : graph_2019

Struktur Tugas

Program minimal memiliki:

a. Vertex and Edge

- Minimal 10 simpul (vertex)

- Minimal 15 sisi (edge)
 - Nama simpul harus merepresentasikan lokasi pada peta yang dibuat
- b. Method yang Wajib Ada
- BFS()
Melakukan pencarian menggunakan algoritma Breadth First Search.
 - DFS()
Melakukan pencarian menggunakan algoritma Depth First Search.
 - displayGraph()
Menampilkan graph pada GUI.
 - displayPath()
Menampilkan jalur yang ditemukan.
 - resetGraph()
Mengembalikan kondisi graph ke keadaan awal.
- c. GUI Program
- GUI minimal memiliki:
1. Area visualisasi graph
 2. ComboBox atau TextField untuk memilih lokasi awal
 3. ComboBox atau TextField untuk memilih lokasi tujuan
 4. Tombol BFS
 5. Tombol DFS
 6. Tombol Reset
 7. Area hasil pencarian
 8. berikan warna berbeda pada node yang telah dikunjungi.
 9. Menampilkan jumlah node yang dieksplorasi.
- d. Ketentuan Graph
1. Setiap mahasiswa wajib membuat graph yang berbeda.
 2. Tidak diperbolehkan menggunakan graph yang sama persis dengan mahasiswa lain.
 3. Graph harus memiliki minimal 10 node dan 15 edge.
 4. Mahasiswa bebas menentukan bentuk dan jenis peta.
 5. Mahasiswa wajib menjelaskan struktur graph, cara kerja BFS dan DFS yang dibuat pada laporan.
- e. Output yang Dikumpulkan
1. Source code Java.
 2. Screenshot program saat dijalankan.
 3. Laporan singkat (PDF) yang berisi:
- f. Deskripsi peta yang dibuat.
- a. Jumlah node dan edge.

- b. Hasil BFS.
- c. Hasil DFS.
- d. Kesimpulan perbandingan BFS dan DFS.

2. Kode program Class SchoolBFSDFS_2511533025

```
package TugasPekan9_2511533025;
import javax.swing.*;
import java.awt.*;
import java.util.*;

public class SchoolBFSDFS_2511533025 extends JFrame {
    private JComboBox<String> cbAwal_3025,
cbTujuan_3025;
    private JTextArea hasilArea_3025;
    private JTextArea graphArea_3025;
    private Graph graph_3025;
    public SchoolBFSDFS_2511533025() {

        graph_3025 = new Graph();
        setTitle("PENCAIRAN JALUR MENGGUNAKAN BFS DAN
DFS");
        setSize(1100, 750);
        setDefaultCloseOperation(EXIT_ON_CLOSE);
        setLocationRelativeTo(null);
        getContentPane().setLayout(new
BorderLayout());

        // ===== HEADER =====

        JPanel header_3025 = new JPanel();
        header_3025.setBackground(new Color(0, 51,
102));
        header_3025.setPreferredSize(new
Dimension(1000, 70));

        JLabel title_3025 = new JLabel(
"PENCAIRAN JALUR MENGGUNAKAN BFS DAN
DFS");

        title_3025.setForeground(Color.WHITE);
        title_3025.setFont(new Font("Arial",
Font.BOLD, 28));
        header_3025.add(title_3025);
        getContentPane().add(header_3025,
BorderLayout.NORTH);

        // ===== PANEL UTAMA
        =====

        JPanel mainPanel_3025 = new JPanel();
        mainPanel_3025.setLayout(null);
```



```

// Lokasi Awal

JLabel lblAwal_3025 = new JLabel("Lokasi
Awal");
lblAwal_3025.setBounds(30, 20, 150, 30);
JLabel titik1_3025 = new JLabel(":");
titik1_3025.setBounds(150, 20, 20, 30);

cbAwal_3025 = new
JComboBox<>(graph_3025.getNodes());
cbAwal_3025.setModel(new
DefaultComboBoxModel(new String[] {"Balai Desa",
"Lapangan Sepak Bola", "Perpustakaan", "Peron Sawit",
"Kantin", "Parkiran", "Rumahku", "Mushola",
"Sekolah", "Puskesmas"}));
cbAwal_3025.setBounds(180, 20, 180, 30);

// Lokasi Tujuan

JLabel lblTujuan_3025 = new JLabel("Lokasi
Tujuan");
lblTujuan_3025.setBounds(30, 60, 150, 30);
JLabel titik2_3025 = new JLabel(":");
titik2_3025.setBounds(150, 60, 20, 30);

cbTujuan_3025 = new
JComboBox<>(graph_3025.getNodes());
cbTujuan_3025.setModel(new
DefaultComboBoxModel(new String[] {"Balai Desa",
"Lapangan Sepak Bola", "Perpustakaan", "Peron Sawit",
"Kantin", "Parkiran", "Rumahku", "Mushola",
"Sekolah", "Puskesmas"}));
cbTujuan_3025.setBounds(180, 60, 180, 30);

// Tombol

JButton btnBFS_3025 = new JButton("BFS");
btnBFS_3025.setBounds(700, 20, 120, 40);
btnBFS_3025.setBackground(new Color(102, 255,
102));

JButton btnDFS_3025 = new JButton("DFS");
btnDFS_3025.setBounds(840, 20, 120, 40);
btnDFS_3025.setBackground(new Color(255, 204,
0));

JButton btnReset_3025 = new JButton("RESET");
btnReset_3025.setBounds(980, 20, 120, 40);
btnReset_3025.setBackground(new Color(255,
102, 102));

// ===== GRAPH =====

```

```

graphArea_3025 = new JTextArea();
graphArea_3025.setEditable(false);
graphArea_3025.setFont(
    new Font("Monospaced",
        Font.BOLD,
        20));

graphArea_3025.setText(
    "\n" +
    "          Rumahku\n" +
    "          |\n" +
    "          ----- Lapangan Sepak
Bola\n" +
    "          |                               |\n" +
    "Perpustakaan ----- Sekolah -----
Kantin\n" +
    "          |          |-----|||---| |---
--| |\n" +
    " Balai Desa--|          Mushola  -|---
Peron Sawit\n" +
    "          |                               |  |--|
|\n" +
    " Puskesmas ----- Parkiran -----
----\n"
);
JScrollPane graphScroll_3025 =
    new JScrollPane(graphArea_3025);
graphScroll_3025.setBorder(
    BorderFactory.createTitledBorder(
        "VISUALISASI GRAPH"));
graphScroll_3025.setBounds(30, 120, 1040,
300);

// ===== HASIL =====

hasilArea_3025 = new JTextArea();
hasilArea_3025.setFont(
    new Font("Monospaced",
        Font.BOLD,
        18));
hasilArea_3025.setEditable(false);
hasilArea_3025.setText(
    "Hasil Pencarian :\n" +
    "Jalur :\n" +
    "Node Dikunjungi :\n" +
    "Jumlah Node Dikunjungi : 0");

JScrollPane hasilScroll_3025 =
    new JScrollPane(hasilArea_3025);

hasilScroll_3025.setBounds(
    30,

```

```

        450,
        1040,
        180);

mainPanel_3025.add(lblAwal_3025);
mainPanel_3025.add(titik1_3025);
mainPanel_3025.add(cbAwal_3025);

mainPanel_3025.add(lblTujuan_3025);
mainPanel_3025.add(titik2_3025);
mainPanel_3025.add(cbTujuan_3025);

mainPanel_3025.add(btnBFS_3025);
mainPanel_3025.add(btnDFS_3025);
mainPanel_3025.add(btnReset_3025);

mainPanel_3025.add(graphScroll_3025);
mainPanel_3025.add(hasilScroll_3025);

getContentPane().add(mainPanel_3025,
BorderLayout.CENTER);

// ===== EVENT BFS
=====

        btnBFS_3025.addActionListener(e -> {
            String start_3025 =

cbAwal_3025.getSelectedItem().toString();
            String goal_3025 =

cbTujuan_3025.getSelectedItem().toString();
            SearchResult result_3025 =
                graph_3025.BFS(start_3025,
goal_3025);
            tampilkanHasil("BFS", result_3025);
        });

// ===== EVENT DFS
=====

        btnDFS_3025.addActionListener(e -> {
            String start_3025 =

cbAwal_3025.getSelectedItem().toString();
            String goal_3025 =

cbTujuan_3025.getSelectedItem().toString();
            SearchResult result_3025 =
                graph_3025.DFS(start_3025,
goal_3025);
            tampilkanHasil("DFS", result_3025);
        });

```

```

// ===== RESET =====

btnReset_3025.addActionListener(e -> {

    hasilArea_3025.setText(
        "Hasil Pencarian :\n" +
        "Jalur :\n" +
        "Node Dikunjungi :\n" +
        "Jumlah Node Dikunjungi : 0");
    });
}
private void tampilkanHasil(
    String metode_3025,
    SearchResult result_3025) {

    hasilArea_3025.setText(

        "Metode : " + metode_3025 + "\n\n" +

        "Jalur :\n" +

        String.join(" -> ",
            result_3025.path_3025)

        +

        "\n\nNode Dikunjungi :\n"

        +

        String.join(" -> ",
            result_3025.visited_3025)

        +

        "\n\nJumlah Node Dikunjungi : "

        +

        result_3025.visited_3025.size()

    );
}

//
=====
// SEARCH RESULT
//
=====

static class SearchResult {
    java.util.List<String> path_3025;
    java.util.List<String> visited_3025;
    SearchResult(

```

```

        java.util.List<String> path_3025,
        java.util.List<String> visited_3025)
    {
        this.path_3025 = path_3025;
        this.visited_3025 = visited_3025;
    }
}

//
=====
// GRAPH
//
=====

static class Graph {
    private Map<String,
        java.util.List<String>> adj =
        new HashMap<>();

    Graph() {
        addEdge("Sekolah",
            "Lapangan Sepak Bola");

        addEdge("Sekolah",
            "Mushola");

        addEdge("Sekolah",
            "Balai Desa");

        addEdge("Sekolah",
            "Peron Sawit");

        addEdge("Sekolah",
            "Kantin");

        addEdge("Sekolah",
            "Perpustakaan");

        addEdge("Lapangan Sepak Bola",
            "Rumahku");

        addEdge("Perpustakaan",
            "Balai Desa");

        addEdge("Balai Desa",
            "Puskesmas");

        addEdge("Puskesmas",
            "Parkiran");

        addEdge("Parkiran",
            "Mushola");

        addEdge("Kantin",

```

```

        "Peron Sawit");

    addEdge("Kantin",
            "Parkiran");

    addEdge("Peron Sawit",
            "Parkiran");

    addEdge("Lapangan Sepak Bola",
            "Perpustakaan");
}

void addEdge(String a_3025, String b_3025) {
    adj.computeIfAbsent(
        a_3025,
        k_3025 -> new
ArrayList<>()).add(b_3025);

    adj.computeIfAbsent(
        b_3025,
        k_3025 -> new
ArrayList<>()).add(a_3025);
}

String[] getNodes() {
    return adj.keySet()
        .toArray(new String[0]);
}

SearchResult BFS(
    String start_3025,
    String goal_3025) {
    Queue<String> queue_3025 =
        new LinkedList<>();
    LinkedHashSet<String> visited_3025 =
        new LinkedHashSet<>();
    Map<String, String> parent_3025 =
        new HashMap<>();

    queue_3025.add(start_3025);
    visited_3025.add(start_3025);
    while (!queue_3025.isEmpty()) {
        String current_3025 =
            queue_3025.poll();

        if (current_3025.equals(goal_3025))
            break;
        for (String next_3025 :
            adj.get(current_3025)) {
            if
(!visited_3025.contains(next_3025)) {
                visited_3025.add(next_3025);
                parent_3025.put(next_3025,
                    current_3025);
                queue_3025.add(next_3025);
            }
        }
    }
}

```

```

    }
    return buildResult(
        goal_3025,
        parent_3025,
        visited_3025);
}
SearchResult DFS(
    String start_3025,
    String goal_3025) {
    Stack<String> stack_3025 =
        new Stack<>();
    LinkedHashSet<String> visited_3025 =
        new LinkedHashSet<>();

    Map<String, String> parent_3025 =
        new HashMap<>();
    stack_3025.push(start_3025);

    while (!stack_3025.isEmpty()) {
        String current_3025 =
            stack_3025.pop();

        if
        (visited_3025.contains(current_3025))
            continue;
        visited_3025.add(current_3025);
        if (current_3025.equals(goal_3025))
            break;
        for (String next_3025 :
            adj.get(current_3025)) {
            if
            (!visited_3025.contains(next_3025)) {
                parent_3025.put(next_3025,
                    current_3025);
                stack_3025.push(next_3025);
            }
        }
    }
    return buildResult(
        goal_3025,
        parent_3025,
        visited_3025);
}
SearchResult buildResult(
    String goal_3025,
    Map<String, String> parent_3025,
    Set<String> visited_3025) {

    java.util.List<String> path_3025 =
        new ArrayList<>();
    String current_3025 = goal_3025;
    path_3025.add(current_3025);

    while
    (parent_3025.containsKey(current_3025)) {

```



```

        current_3025 =
parent_3025.get(current_3025);
        path_3025.add(current_3025);
    }
    Collections.reverse(path_3025);
    return new SearchResult(
        path_3025,
        new ArrayList<>(visited_3025));
    }
}
public static void main(String[] args) {
    SwingUtilities.invokeLater(() -> {
        new SchoolBFSDFS_2511533025()
            .setVisible(true);
    });
}
}

```

3. Gambar kode program dan output program.

1. Gambar Class SchoolBFSDFS_2511533025

```

> 2025_prakalpro_C_2511533025 [repository master]
> PraksD2026_C_2511533025 [PraksD2026_C_2511533025]
> JRE System Library [JavaSE-21]
  > src
    > pekan1_2511533025
    > pekan2_2511533025
    > pekan3_2511533025
    > pekan4_2511533025
    > pekan5_2511533025
    > pekan6_2511533025
    > pekan7_2511533025
    > pekan8_2511533025
    > tugasPekan1_2511533025
    > tugasPekan2_2511533025
    > tugasPekan3_2511533025
    > tugasPekan4_2511533025
    > tugasPekan5_2511533025
    > tugasPekan6_2511533025
    > tugasPekan7_2511533025
    > tugasPekan8_2511533025
    > tugasPekan9_2511533025
    > UjiCoba_2511533025
    > module-info.java

1 package TugasPekan9_2511533025;
2 import javax.swing.*;
3
4
5
6 public class SchoolBFSDFS_2511533025 extends JFrame {
7     private JComboBox<String> cbAwal_3025, cbTujuan_3025;
8     private JTextArea hasilArea_3025;
9     private JTextArea graphArea_3025;
10    private Graph graph_3025;
11    public SchoolBFSDFS_2511533025() {
12
13        graph_3025 = new Graph();
14        setTitle("PENCAIRAN JALUR MENGGUNAKAN BFS DAN DFS");
15        setSize(1100, 750);
16        setDefaultCloseOperation(EXIT_ON_CLOSE);
17        setLocationRelativeTo(null);
18        getContentPane().setLayout(new BorderLayout());
19
20        // ===== HEADER =====
21
22        JPanel header_3025 = new JPanel();
23        header_3025.setBackground(new Color(0, 51, 102));
24        header_3025.setPreferredSize(new Dimension(1000, 70));
25
26        JLabel title_3025 = new JLabel(
27            "PENCAIRAN JALUR MENGGUNAKAN BFS DAN DFS");
28
29        title_3025.setForeground(Color.WHITE);
30        title_3025.setFont(new Font("Arial", Font.BOLD, 28));
31        header_3025.add(title_3025);
32        getContentPane().add(header_3025, BorderLayout.NORTH);
33
34        // ===== PANEL UTAMA =====
35
36        JPanel mainPanel_3025 = new JPanel();
37        mainPanel_3025.setLayout(null);
38
39
40        // Lokasi Awal
41        JLabel lblAwal_3025 = new JLabel("Lokasi Awal");
42        lblAwal_3025.setBounds(30, 20, 150, 30);
43        JLabel titiki_3025 = new JLabel("");
44        titiki_3025.setBounds(150, 20, 20, 30);
45
46        cbAwal_3025 = new JComboBox<>(graph_3025.getNodes());
47        cbAwal_3025.setModel(new DefaultComboBoxModel<new String[] { "Balai Desa", "Lapangan Sepak Bola", "Perpustakaan", "Peron Sawit", "Kantin" }
48        cbAwal_3025.setBounds(180, 20, 180, 30);
49
50        // Lokasi Tujuan
51
52        JLabel lblTujuan_3025 = new JLabel("Lokasi Tujuan");
53        lblTujuan_3025.setBounds(30, 60, 150, 30);
54        JLabel titik2_3025 = new JLabel("");
55        titik2_3025.setBounds(150, 60, 20, 30);
56
57        cbTujuan_3025 = new JComboBox<>(graph_3025.getNodes());
58        cbTujuan_3025.setModel(new DefaultComboBoxModel<new String[] { "Balai Desa", "Lapangan Sepak Bola", "Perpustakaan", "Peron Sawit", "Kantin" }
59        cbTujuan_3025.setBounds(180, 60, 180, 30);
60
61        // Tombol
62
63        JButton btnBFS_3025 = new JButton("BFS");
64        btnBFS_3025.setBounds(700, 20, 120, 40);
65        btnBFS_3025.setBackground(new Color(102, 255, 102));
66
67        JButton btnDFS_3025 = new JButton("DFS");
68        btnDFS_3025.setBounds(840, 20, 120, 40);
69        btnDFS_3025.setBackground(new Color(255, 204, 0));
70
71        JButton btnReset_3025 = new JButton("RESET");
72        btnReset_3025.setBounds(980, 20, 120, 40);
73        btnReset_3025.setBackground(new Color(255, 102, 102));
74
75        // ===== GRAPH =====
76
77        graphArea_3025 = new JTextArea();
78        graphArea_3025.setEditable(false);
79        graphArea_3025.setFont(
80            new Font("Monospaced",
81                Font.BOLD,
82                20));
83

```

```

83
84 graphArea_3025.setText(
85
86     "\n" +
87         "        Rumahku\n" +
88         "        | \n" +
89         "        ----- Lapangan Sepak Bola\n" +
90         "        | \n" +
91         "Perpustakaan ----- Sekolah ----- Kantin\n" +
92         "        | | | | | \n" +
93         "Balai Desa----- Mushola ----- Peron Sawit\n" +
94         "        | | | | | \n" +
95         "Puskesmas ----- Parkiran ----- \n"
96
97 );
98 JScrollPane graphScroll_3025 =
99     new JScrollPane(graphArea_3025);
100 graphScroll_3025.setBorder(
101     BorderFactory.createTitledBorder(
102         "VISUALISASI GRAPH");
103 graphScroll_3025.setBounds(30, 120, 1040, 300);
104
105 // ===== HASIL =====
106
107 hasilArea_3025 = new JTextArea();
108 hasilArea_3025.setFont(
109     new Font("Monospaced",
110         Font.BOLD,
111         18));
112 hasilArea_3025.setEditable(false);
113 hasilArea_3025.setText(
114     "Hasil Pencarian :\n" +
115     "Jalur :\n" +
116     "Node Dikunjungi :\n" +
117     "Jumlah Node Dikunjungi : 0");
118
119 JScrollPane hasilScroll_3025 =
120     new JScrollPane(hasilArea_3025);
121
122 hasilScroll_3025.setBounds(
123     30,
124     450,
125     1040,
126     180);
127
128 mainPanel_3025.add(lblAwal_3025);
129 mainPanel_3025.add(titik1_3025);
130 mainPanel_3025.add(cbAwal_3025);
131
132 mainPanel_3025.add(lblTujuan_3025);
133 mainPanel_3025.add(titik2_3025);
134 mainPanel_3025.add(cbTujuan_3025);
135
136 mainPanel_3025.add(btnBFS_3025);
137 mainPanel_3025.add(btnDFS_3025);
138 mainPanel_3025.add(btnReset_3025);
139
140 mainPanel_3025.add(graphScroll_3025);
141 mainPanel_3025.add(hasilScroll_3025);
142
143 getContentPane().add(mainPanel_3025, BorderLayout.CENTER);
144
145 // ===== EVENT BFS =====
146
147 btnBFS_3025.addActionListener(e -> {
148     String start_3025 =
149         cbAwal_3025.getSelectedItem().toString();
150     String goal_3025 =
151         cbTujuan_3025.getSelectedItem().toString();
152     SearchResult result_3025 =
153         graph_3025.BFS(start_3025, goal_3025);
154     tampilkanHasil("BFS", result_3025);
155 });
156
157 // ===== EVENT DFS =====
158
159 btnDFS_3025.addActionListener(e -> {
160     String start_3025 =
161         cbAwal_3025.getSelectedItem().toString();
162     String goal_3025 =
163         cbTujuan_3025.getSelectedItem().toString();
164     SearchResult result_3025 =
165         graph_3025.DFS(start_3025, goal_3025);
166     tampilkanHasil("DFS", result_3025);
167 });
168

```

```

168
169 // ===== RESET =====
170
171 btnReset_3025.addActionListener(e -> {
172
173     hasilArea_3025.setText(
174         "Hasil Pencarian :\n" +
175         "Jalur :\n" +
176         "Node Dikunjungi :\n" +
177         "Jumlah Node Dikunjungi : 0");
178     });
179 }
180 private void tampilkanHasil(
181     String metode_3025,
182     SearchResult result_3025) {
183
184     hasilArea_3025.setText(
185
186         "Metode : " + metode_3025 + "\n\n" +
187
188         "Jalur :\n" +
189
190         String.join(" -> ",
191             result_3025.path_3025)
192
193         +
194
195         "\n\nNode Dikunjungi :\n"
196
197         +
198
199         String.join(" -> ",
200             result_3025.visited_3025)
201
202         +
203
204         "\n\nJumlah Node Dikunjungi : "
205
206         +
207
208         result_3025.visited_3025.size()
209
210     );
211 }
212

```

```

213 // =====
214 // SEARCH RESULT
215 // =====
216
217 static class SearchResult {
218     java.util.List<String> path_3025;
219     java.util.List<String> visited_3025;
220     SearchResult(
221         java.util.List<String> path_3025,
222         java.util.List<String> visited_3025) {
223         this.path_3025 = path_3025;
224         this.visited_3025 = visited_3025;
225     }
226 }
227
228 // =====
229 // GRAPH
230 // =====
231
232 static class Graph {
233     private Map<String,
234         java.util.List<String>> adj =
235         new HashMap<>();
236
237     Graph() {
238         addEdge("Sekolah",
239             "Lapangan Sepak Bola");
240
241         addEdge("Sekolah",
242             "Mushola");
243
244         addEdge("Sekolah",
245             "Balai Desa");
246
247         addEdge("Sekolah",
248             "Peron Sawit");
249
250         addEdge("Sekolah",
251             "Kantin");
252
253         addEdge("Sekolah",
254             "Perpustakaan");
255
256         addEdge("Lapangan Sepak Bola",
257             "Rumahku");
258
259         addEdge("Perpustakaan",
260             "Balai Desa");
261
262         addEdge("Balai Desa",
263             "Puskesmas");
264
265         addEdge("Puskesmas",
266             "Parkiran");
267
268         addEdge("Parkiran",
269             "Mushola");
270
271         addEdge("Kantin",
272             "Peron Sawit");
273
274         addEdge("Kantin",
275             "Parkiran");
276
277         addEdge("Peron Sawit",
278             "Parkiran");
279
280         addEdge("Lapangan Sepak Bola",
281             "Perpustakaan");
282     }
283     void addEdge(String a_3025, String b_3025) {
284         adj.computeIfAbsent(
285             a_3025,
286             k_3025 -> new ArrayList<>()).add(b_3025);
287
288         adj.computeIfAbsent(
289             b_3025,
290             k_3025 -> new ArrayList<>()).add(a_3025);
291     }
292     String[] getNodes() {
293         return adj.keySet()
294             .toArray(new String[0]);
295     }

```



```

296 SearchResult BFS(
297     String start_3025,
298     String goal_3025) {
299     Queue<String> queue_3025 =
300         new LinkedList<>();
301     LinkedHashSet<String> visited_3025 =
302         new LinkedHashSet<>();
303     Map<String, String> parent_3025 =
304         new HashMap<>();
305
306     queue_3025.add(start_3025);
307     visited_3025.add(start_3025);
308     while (!queue_3025.isEmpty()) {
309         String current_3025 =
310             queue_3025.poll();
311
312         if (current_3025.equals(goal_3025))
313             break;
314         for (String next_3025 :
315             adj.get(current_3025)) {
316             if (!visited_3025.contains(next_3025)) {
317                 visited_3025.add(next_3025);
318                 parent_3025.put(next_3025,
319                     current_3025);
320                 queue_3025.add(next_3025);
321             }
322         }
323     }
324     return buildResult(
325         goal_3025,
326         parent_3025,
327         visited_3025);
328 }
329
330 SearchResult DFS(
331     String start_3025,
332     String goal_3025) {
333     Stack<String> stack_3025 =
334         new Stack<>();
335     LinkedHashSet<String> visited_3025 =
336         new LinkedHashSet<>();
337
338     Map<String, String> parent_3025 =
339         new HashMap<>();
340     stack_3025.push(start_3025);
341     while (!stack_3025.isEmpty()) {
342         String current_3025 =
343             stack_3025.pop();
344         if (visited_3025.contains(current_3025))
345             continue;
346         visited_3025.add(current_3025);
347         if (current_3025.equals(goal_3025))
348             break;
349         for (String next_3025 :
350             adj.get(current_3025)) {
351             if (!visited_3025.contains(next_3025)) {
352                 parent_3025.put(next_3025,
353                     current_3025);
354                 stack_3025.push(next_3025);
355             }
356         }
357     }
358     return buildResult(
359         goal_3025,
360         parent_3025,
361         visited_3025);
362 }

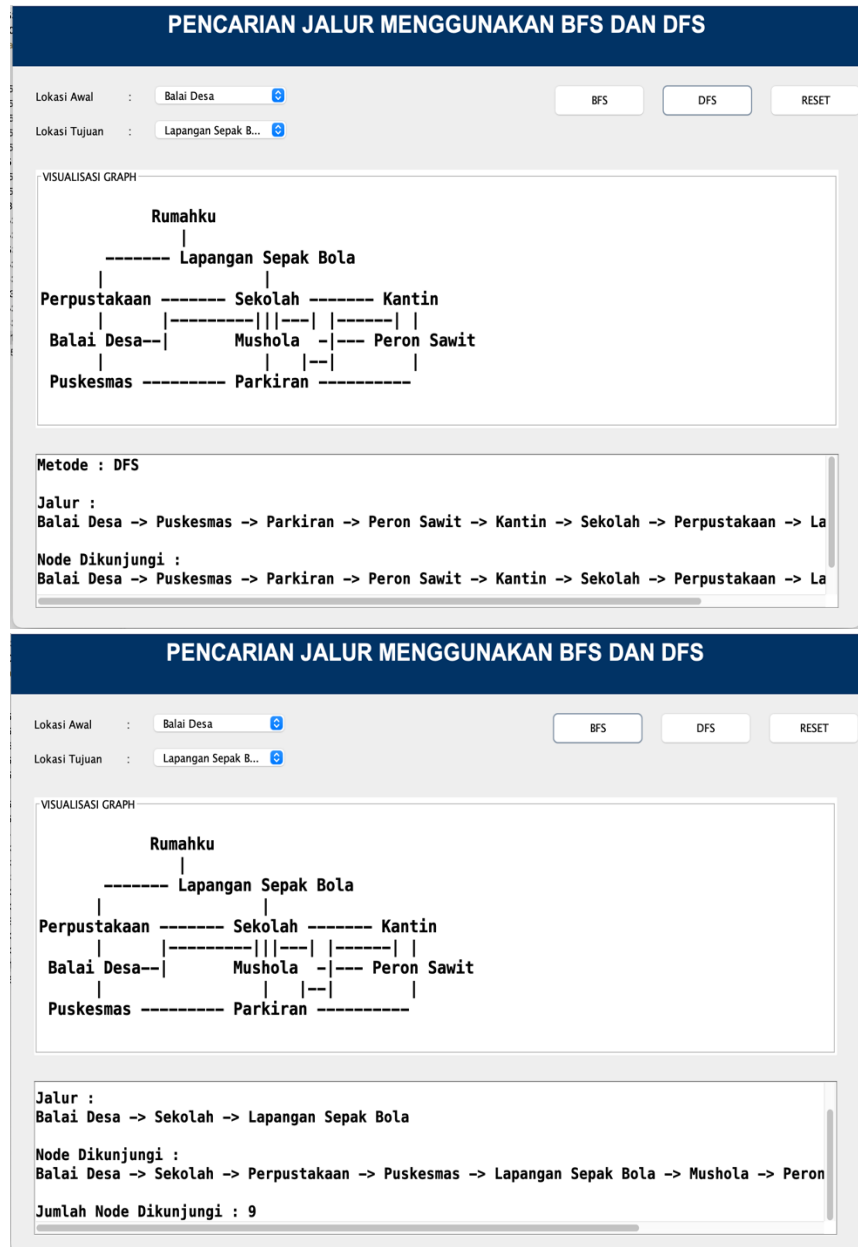
```

```

363     SearchResult buildResult(
364         String goal_3025,
365         Map<String, String> parent_3025,
366         Set<String> visited_3025) {
367
368         java.util.List<String> path_3025 =
369             new ArrayList<>();
370         String current_3025 = goal_3025;
371         path_3025.add(current_3025);
372
373         while (parent_3025.containsKey(current_3025)) {
374             current_3025 =
375                 parent_3025.get(current_3025);
376             path_3025.add(current_3025);
377         }
378         Collections.reverse(path_3025);
379         return new SearchResult(
380             path_3025,
381             new ArrayList<>(visited_3025));
382     }
383 }
384 public static void main(String[] args) {
385     SwingUtilities.invokeLater(() -> {
386         new SchoolBFSDFS_2511533025()
387             .setVisible(true);
388     });
389 }
390 }
391 }

```

2. Gambar output code program Class SchoolBFSDFS_2511533025



4. Penjelasan tentang kode program yang dibuat

a. Deskripsi peta yang dibuat

Peta yang digunakan dalam program merepresentasikan lingkungan sekolah yang terdiri dari beberapa lokasi yang saling terhubung dalam bentuk graph. Titik pusat peta adalah Sekolah. Di belakang sekolah terdapat Lapangan Sepak Bola, dan di belakang lapangan terdapat Rumahku. Di depan sekolah terdapat Mushola, di sebelah kiri terdapat Balai Desa, dan di sebelah kanan terdapat Peron Sawit. Selain itu terdapat lokasi pendukung lainnya yaitu Perpustakaan, Kantin, Puskesmas, dan Parkiran.

b. Jumlah Node dan Edge

Node (Vertex)

No Nama Node

- | | |
|----|---------------------|
| 1 | Sekolah |
| 2 | Lapangan Sepak Bola |
| 3 | Rumahku |
| 4 | Mushola |
| 5 | Balai Desa |
| 6 | Peron Sawit |
| 7 | Kantin |
| 8 | Perpustakaan |
| 9 | Puskesmas |
| 10 | Parkiran |

Jumlah Node = 10

Edge (Sisi)

No Edge

- | | |
|----|------------------------------------|
| 1 | Sekolah – Lapangan Sepak Bola |
| 2 | Sekolah – Mushola |
| 3 | Sekolah – Balai Desa |
| 4 | Sekolah – Peron Sawit |
| 5 | Sekolah – Kantin |
| 6 | Sekolah – Perpustakaan |
| 7 | Lapangan Sepak Bola – Rumahku |
| 8 | Perpustakaan – Balai Desa |
| 9 | Balai Desa – Puskesmas |
| 10 | Puskesmas – Parkiran |
| 11 | Parkiran – Mushola |
| 12 | Kantin – Peron Sawit |
| 13 | Kantin – Parkiran |
| 14 | Peron Sawit – Parkiran |
| 15 | Lapangan Sepak Bola – Perpustakaan |

Jumlah Edge = 15

c. Hasil BFS

Contoh pencarian:

Start : Sekolah

Goal : Rumahku

Proses BFS

BFS menelusuri node berdasarkan tingkat kedekatan (level).

Urutan node yang dikunjungi:

Sekolah

Lapangan Sepak Bola

Mushola

Balai Desa

Peron Sawit
Kantin
Perpustakaan
Rumahku

Jalur yang ditemukan: Sekolah -> Lapangan Sepak Bola ->
Rumahku
Jumlah node yang dieksplorasi: 8 node

d. Hasil DFS

Contoh pencarian:

Start : Sekolah

Goal : Rumahku

Proses DFS

DFS menelusuri satu cabang sedalam mungkin sebelum kembali ke cabang lain.

Urutan node yang dikunjungi:

Sekolah

Perpustakaan

Lapangan

Rumahku

Jalur yang ditemukan: Sekolah -> Perpustakaan -> Lapangan
Sepak Bola -> Rumahku

Jumlah node yang dieksplorasi: 4 node

Catatan: Urutan DFS dapat berbeda tergantung urutan tetangga (adjacency list) pada program.

e. Kesimpulan perbandingan BFS dan DFS

Aspek	BFS	DFS
Metode pencarian	Per level	Sedalam mungkin
Struktur data	Queue	Stack
Jalur terpendek	Ya	Tidak selalu
Memori	Lebih besar	Lebih kecil
Kecepatan menemukan solusi		
BFS: Baik untuk jalur terdekat		
DFS: Baik untuk pencarian mendalam		
Hasil pada kasus ini		
BFS: Sekolah → Lapangan → Rumahku		
DFS: Sekolah → Perpustakaan → Lapangan → Rumahku		

BFS lebih cocok digunakan ketika tujuan utama adalah menemukan jalur terpendek karena algoritma akan memeriksa seluruh node pada level yang sama terlebih dahulu. Sementara itu,

DFS lebih cocok digunakan untuk eksplorasi graph yang dalam karena akan menelusuri satu jalur hingga akhir sebelum kembali ke node sebelumnya. Pada studi kasus peta sekolah ini, BFS berhasil menemukan jalur terpendek dari Sekolah menuju Rumahku, sedangkan DFS menemukan jalur yang valid namun tidak selalu merupakan jalur terpendek.

f. Struktur Graph dan cara kerja BFS dan DFS

1. Struktur Graph

Graph adalah struktur data yang terdiri dari:

Vertex (Node) → merepresentasikan lokasi.

Edge (Sisi) → merepresentasikan hubungan antar lokasi.

Pada program ini:

$V = \{\text{Sekolah, Lapangan, Rumahku, Mushola,}$

$\text{Balai Desa, Peron Sawit, Kantin,}$

$\text{Perpustakaan, Pos Satpam, Parkiran}\}$

$E = 15$ sisi penghubung antar lokasi

Graph yang digunakan merupakan Undirected Graph, artinya hubungan antar lokasi dapat dilalui dua arah.

Contoh:

Sekolah <--> Lapangan Sepak Bola

Sekolah <--> Mushola

Sekolah <--> Balai Desa

2. Cara kerja BFS

BFS menggunakan struktur data Queue (Antrian).

Langkah-langkah:

1. Masukkan node awal ke Queue.
2. Kunjungi node pertama.
3. Masukkan seluruh tetangganya ke Queue.
4. Ambil node berikutnya dari Queue.
5. Ulangi hingga node tujuan ditemukan.

Contoh:

Start = Sekolah

Queue:

[Sekolah]

↓

[Lapangan, Mushola,
Balai Desa, Peron Sawit,
Kantin, Perpustakaan]

↓

Rumahku ditemukan

Karena BFS mencari per level, maka jalur yang ditemukan merupakan jalur terpendek.

3. Cara kerja DFS

DFS menggunakan struktur data Stack (Tumpukan).

Langkah-langkah:

Masukkan node awal ke Stack.

Ambil node teratas.

Kunjungi node tersebut.

Pilih satu tetangga dan lanjutkan sedalam mungkin.

Jika buntu, kembali ke node sebelumnya (backtracking).

Ulangi sampai tujuan ditemukan.

Contoh:

Sekolah

↓

Perpustakaan

↓

Lapangan

↓

Rumahku

DFS dapat menemukan tujuan lebih cepat pada beberapa kasus, tetapi tidak menjamin jalur yang diperoleh adalah jalur terpendek.